POLYGLOT WORKSHOP

DEEP LEARNING PERSPECTIVE ON HPC

Tutors
Nicholas Kluge

TRA SUSTAINABLE FUTURES

UNIVERSITÄT BONN

AGENDA

1. Why HPC matters for LLMs?

✓ Why Should LLM People Care About HPC?

2. Inside a GPU

✓ What is a GPU?

✓ GPU Anatomy

✓ FLOPs and Precision

✓ Model FLOP Utilization

✓ Memory Hierarchy

✓ Memory-Bound vs Compute-Bound

AGENDA

3. Outside the GPU

- ✓ Communication: CPU ↔ GPU
- ✓ Communication: GPU ↔ GPU
- ✓ Communication: Multi-Node
- ✓ Communication: Troubleshooting

4. Planning and Heuristics

- ✓ Hardware Reliability for Long Runs
- ✓ Sizing Your Training Run
- ✓ Rules of Thumb for DL on GPUs

WHY HPC MATTERS FOR LLMS

And Why Should LLM People Care About HPC?

WHY INFRASTRUCTURE MATTERS FOR LLMS

- Most LLM bottlenecks are **not algorithmic** — they are **infrastructure bottlenecks.**
- GPUs don’t train models — ***systems do!***
- Meanwhile, understanding infrastructure helps you:

  ✔ Diagnose problems
  ✔ Reason about scaling limits
  ✔ Being able to understand and communicate with your cluster admins!

Infrastructure **is not solved; it's just abstracted away.**

Tazi, N., Mom, F., Zhao, H., Nguyen, P., Mekkouri, M., Werra, L., & Wolf, T. (2025). *The Ultra-Scale Playbook: Training LLMs on GPU Clusters*.

Ben Allal, L., Tunstall, L., Tazi, N., Bakouch, E., Beeching, E., Patiño, C. M., Fourrier, C., Frere, T., Lozhkov, A., Raffel, C., Werra, L. von, & Wolf, T. (2025). *The Smol Training Playbook: Efficient LLM Training Recipes for Small-Scale GPU Clusters*.

INSIDE A GPU

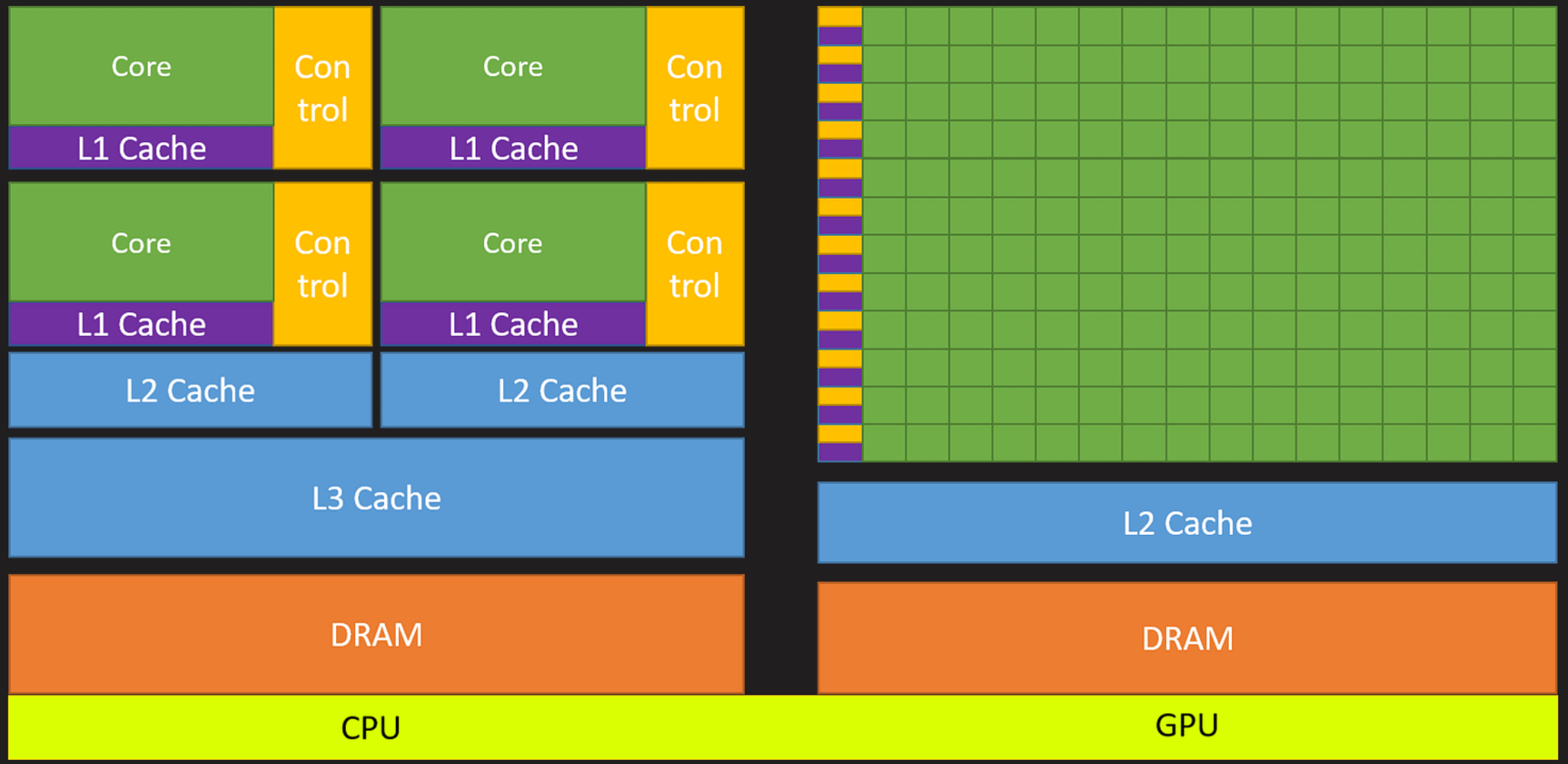
The Throughput Machine

WHAT IS A GPU

- ✓ A GPU is fundamentally a **massively parallel processor optimised for throughput over latency**. Unlike CPUs, which excel at executing a few complex instruction streams quickly, GPUs achieve performance by executing **thousands of simple operations simultaneously**.

- ✓ The key to understanding GPU performance lies in recognising that it's not just about raw compute power, it's about the **interplay between computation and data movement**. A GPU can have teraflops of theoretical compute, but if data **can't reach the compute units fast enough, that potential goes unused**.

💡 ***FLOPs are cheap. Memory is expensive.***



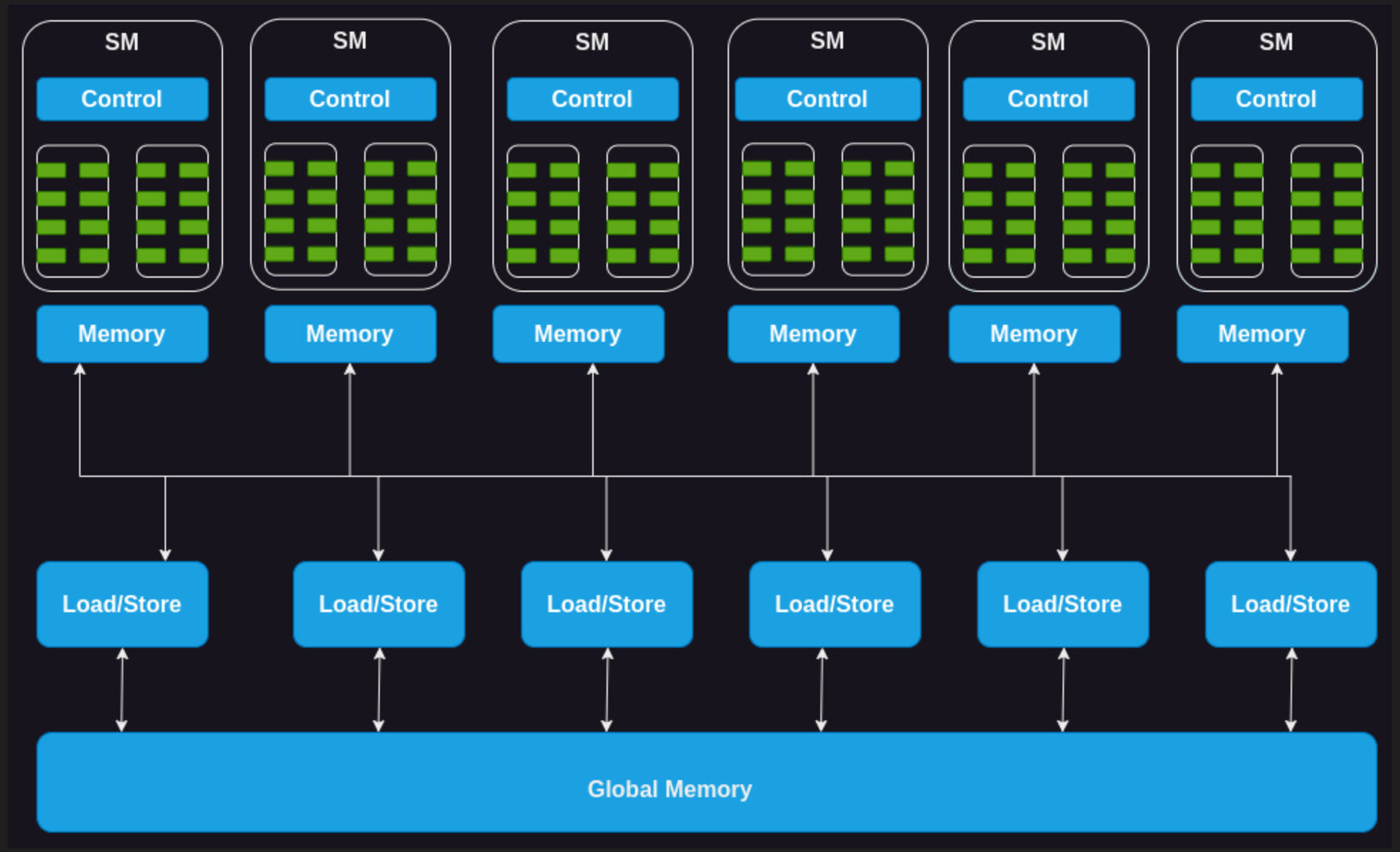
Source → CUDA C++ Docs

GPU ANATOMY

GPUs have a very hierarchical organization. On the compute side, a GPU consists of an array of compute units called ***streaming multiprocessors (SMs)***. Each SM contains and controls a set of streaming processors, also known as *cores*.

- ✔ **CUDA cores (scalar math)**
- ✔ **Tensor Cores (matrix math)**
- ✔ **NVIDIA H100 → 132 SMs → 128 cores per SM → ~17k cores**

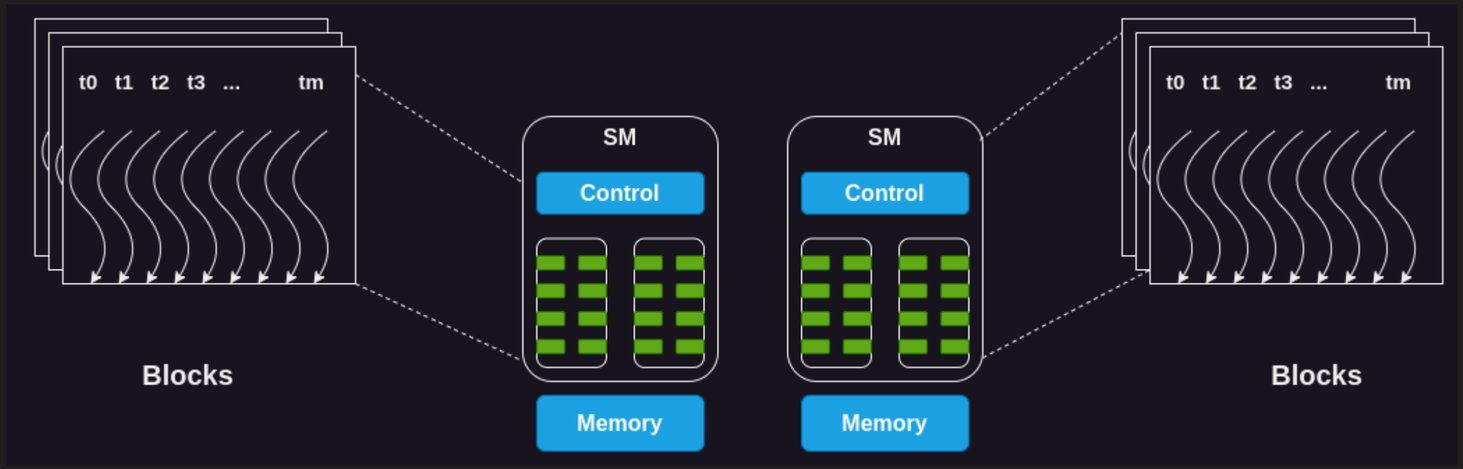
At the highest level, a GPU therefore performs **two essential tasks**: (1) move and store data (**the memory system**), and (2) do work with the data (**the compute pipelines**).



Source → CUDA C++ Docs

GPU ANATOMY

Each SM operates independently, executing groups of **32 threads called warps in lockstep**. To help with this, the SMs rely on another component, the **warp schedulers**: by balancing instructions across different warps, they enable the SM to *“hide latency”* by switching between warps when one is held up. This SIMT **(Single Instruction, Multiple Thread)** execution model means all threads in a warp execute the same instruction simultaneously on different data.



Source → CUDA C++ Docs

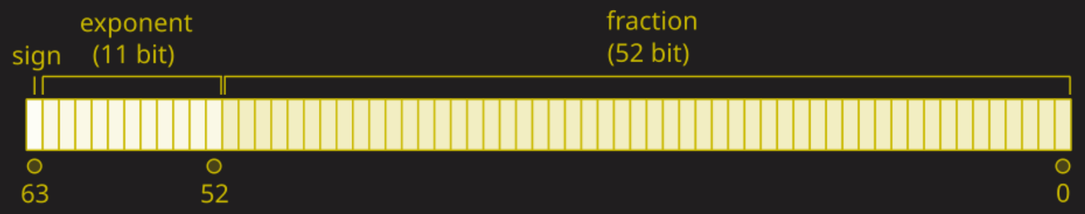
FLOPS AND PRECISION

- GPUs measure performance in FLOPs (floating-point operations per second). A FLOP is a single arithmetic operation, typically a floating-point number addition like `(a + b)`, and modern GPUs can execute trillions of these per second (TFLOPs).

  ✔ **FLOP = one floating-point operation**

  ✔ GPU vendors advertise **peak TFLOPs**, but these depend on the **level of precision** used

Precision matters significantly when discussing FLOPs. Tensor Cores can operate at different precisions (**FP64, FP32, FP16/BF16, FP8, FP4**), which produce very different results (a reminder on floating-point numbers).



Source → Wikipedia

FLOPS AND PRECISION

In general, **lower precision** means **higher throughput** (primarily because of **less memory movement**). Modern training of LLMs is (almost) exclusively done with mixed precision (**FP32/BF16**), with new regimes of lower precision becoming increasingly more viable (**FP8/FP4**).

Precision \ GPU Type	A100	H100	H200	B100	B200
FP64	9.7	34	34	40	40
FP32	19.5	67	67	80	80
FP16/BF16	300	990	990	1750	2250
FP8	-	3960	3960	4500	5000
FP4	-	-	-	9000	10000*

*More operations per **watt** and per **second**.

MODEL FLOP UTILIZATION

- ✓ Those theoretical peak FLOPs represent the ***maximum computational throughput achievable under ideal conditions***, when all compute units are fully utilized, and data is readily available. In practice, actual performance depends heavily on how well your workload can keep the compute units fed with data and whether your operations can be efficiently mapped to the available hardware.

- ✓ Hence, comes the question: ***"How much of the GPU throughput are we getting in our very non-ideal conditions?"***

- ✓ **Model FLOP Utilization (MFU)** is a performance efficiency metric that measures how well a model training run utilizes the available peak compute of the hardware. If, while training your model, you achieve an MFU of 40%, that means you are using 40% of the GPU's maximum theoretical compute capacity. This metric was proposed in Google's PaLM paper.

MODEL FLOP UTILIZATION

To compute MFU, you need:

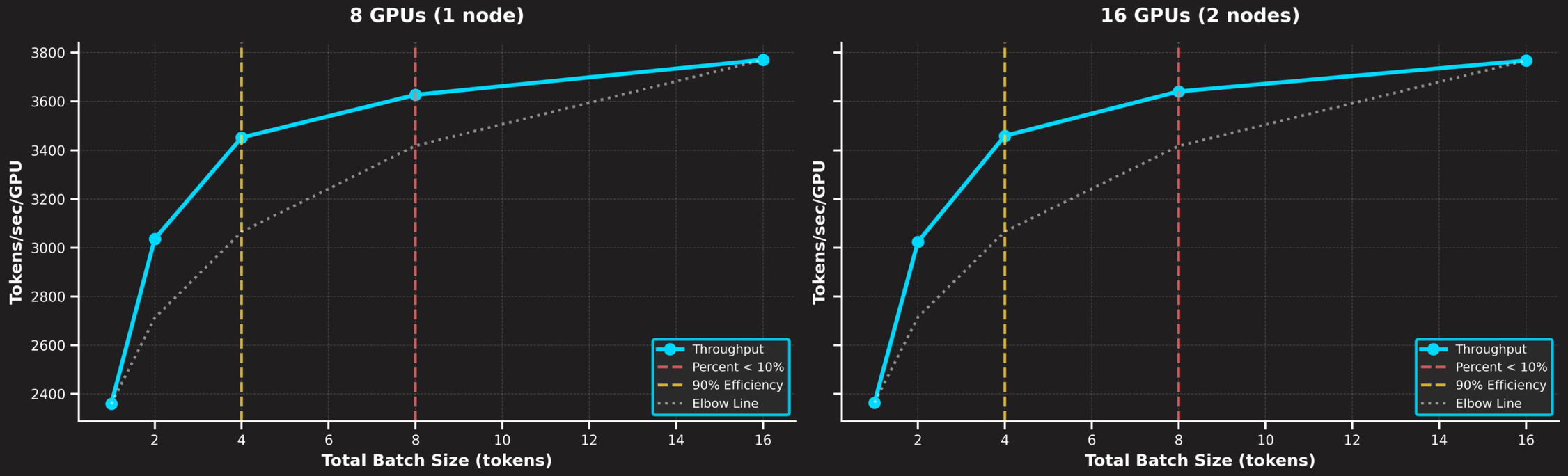
- ✔ **Peak FLOPs of the hardware** (e.g., 312 TFLOPs)
- ✔ **Model FLOPs per iteration** (operations per forward + backward pass)
- ✔ **Achieved FLOPs per second** (model FLOPs divided by actual training step time)
- ✔ **MFU = Achieved FLOPs / Peak FLOPs**

MFU Calculator

Must read: Adam Casson (2023). *Transformer FLOPs*.

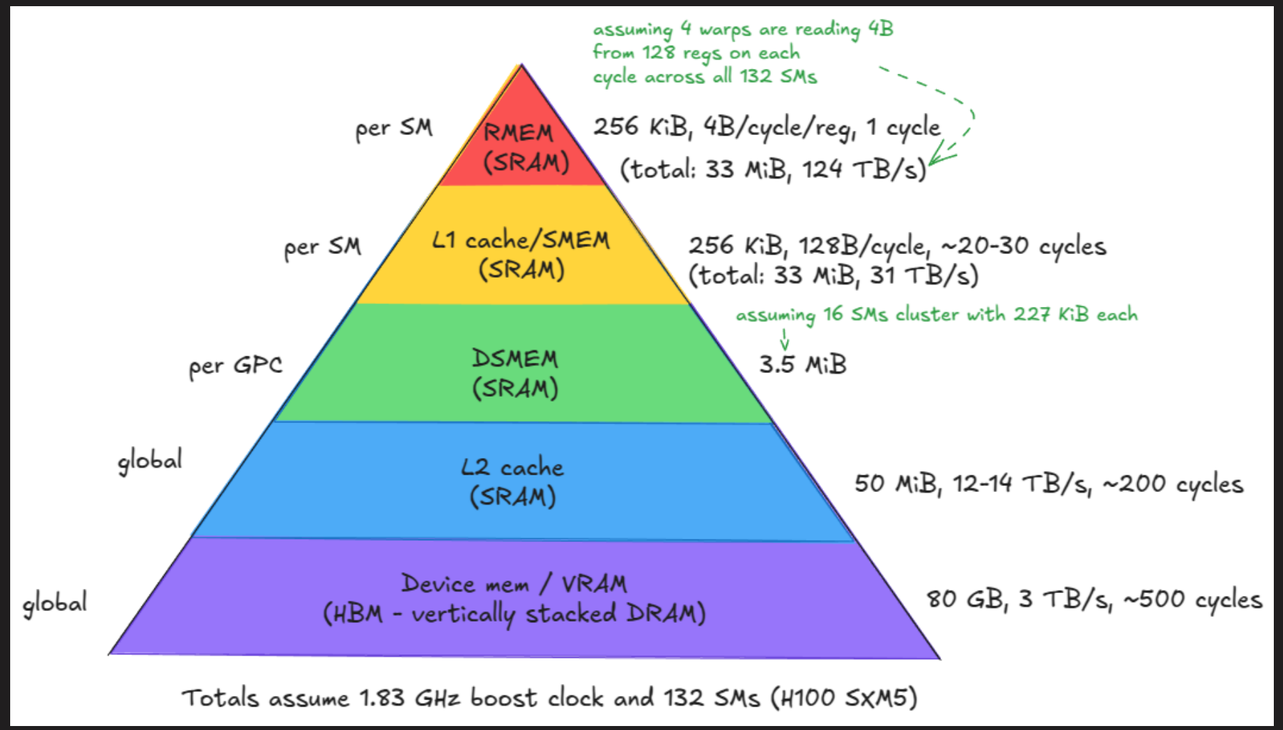
```python
# 1. Hardware peak FLOPs
P = 300e12  # 300 TFLOPs (A100)

# 2. Model configuration
C = 1100048384   # number of trainable parameters
L = 22           # number of layers
H = 32           # number of attention heads
Q = 64           # hidden size per attention head
T = 2048         # max sequence length (position embeddings)

# FLOPs per token (rule of thumb: 6*C + 12*L*H*Q*T)
# This heuristic also comes from the [PALM paper](https://arxiv.org/abs/2204.02311)
# The 6 accounts for backward and forward passes,
# while the 12 accounts for transformer-specific operations (e.g., multi-head attention)
flops_per_token = 6*C + 12*L*H*Q*T

# 3. Training runtime info
dt = 1.4        # step time (s)
batch_size = 16 # batch size

# FLOPs per forward+backward pass
flops_per_fwdbwd = flops_per_token * T

# FLOPs per iteration (batch_size sequences processed per step)
flops_per_iter = flops_per_fwdbwd * batch_size

# Achieved FLOPs per second
flops_achieved = flops_per_iter / dt

# MFU ratio
mfu = flops_achieved / P

print(f"MFU: {mfu * 100:.2f}%")
```

MODEL FLOPS UTILIZATION



However, it is essential to understand that, in terms of model training, **MFU is not everything**. We also care about *how long* it will take to train the model. Sometimes, **more GPUs with a more diluted MFU are a better alternative**.

MEMORY HIERARCHY



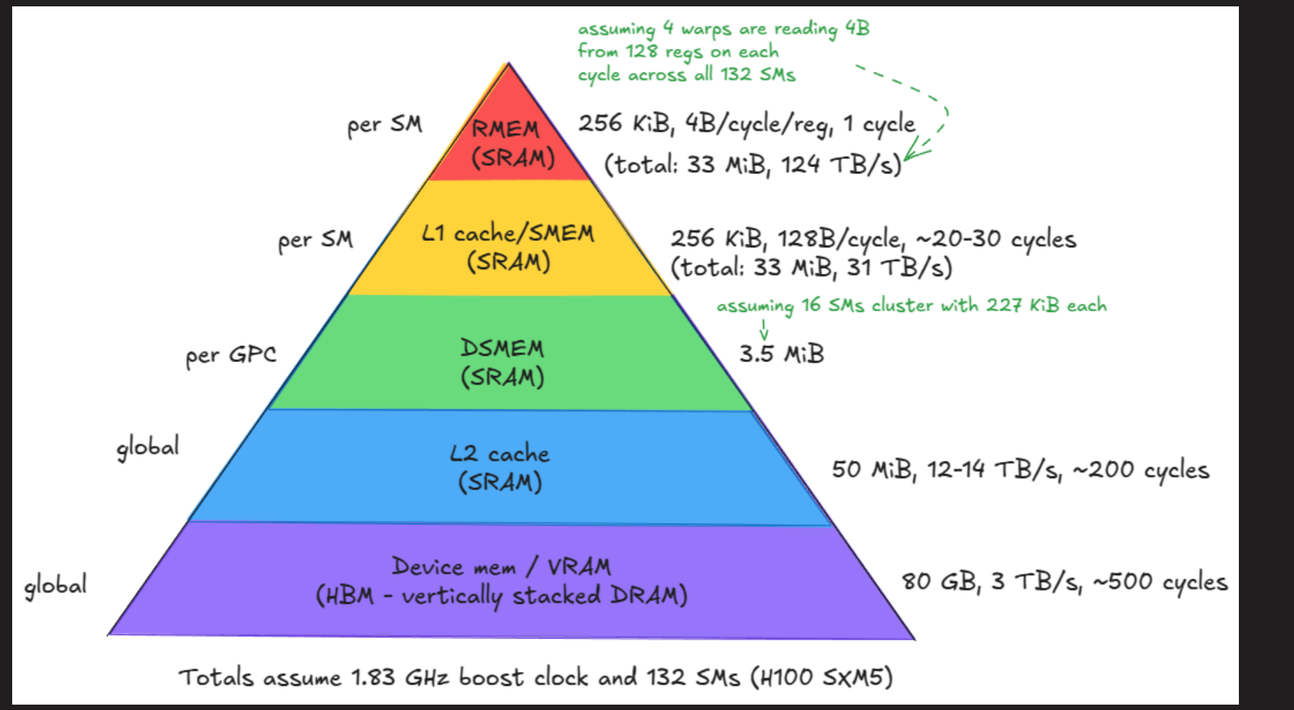
Source → Aleksa Gordić (MatMul blog)

GPUs organize memory in a **hierarchy from fast-but-small** (*registers, shared memory*) to **slow-but-large** (*HBM main memory*). Understanding this hierarchy is critical because modern AI is often memory-bound: ***the bottleneck is moving data, not computing on it.***

MEMORY HIERARCHY

This hierarchy exists because SRAM is fast but physically **large and expensive**, while DRAM is dense and **cheap but slower**. The result: fast memory comes in small quantities near compute cores, backed by progressively larger pools of slower memory farther away.

A lot of **optimization** comes from understanding and leveraging this hierarchy.



Source → Aleksa Gordić (MatMul blog)

"load from memory" → "multiply by itself twice" → "write to memory"* takes essentially the same time as *"load from memory" → "multiply by itself once" → "write to memory"

Must read: Making Deep Learning Go Brrrr From First Principles)

MEMORY HIERARCHY

Flash Attention⚡ is a **case study in memory optimization**. Standard attention implementations are memory-bound because they **materialize the full attention matrix in HBM**:

- ✔ Compute Q @ K^T → write N×N attention scores to HBM
- ✔ Apply Softmax → read from HBM, compute, write back to HBM
- ✔ Multiply by V → read attention scores from HBM …

Flash Attention achieves its 2-4× speedup by **fusing these operations and keeping intermediate results in SRAM!**

Dao, T., Fu, D., Ermon, S., Rudra, A., & Ré, C. (2022). Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, *35*, 16344-16359.

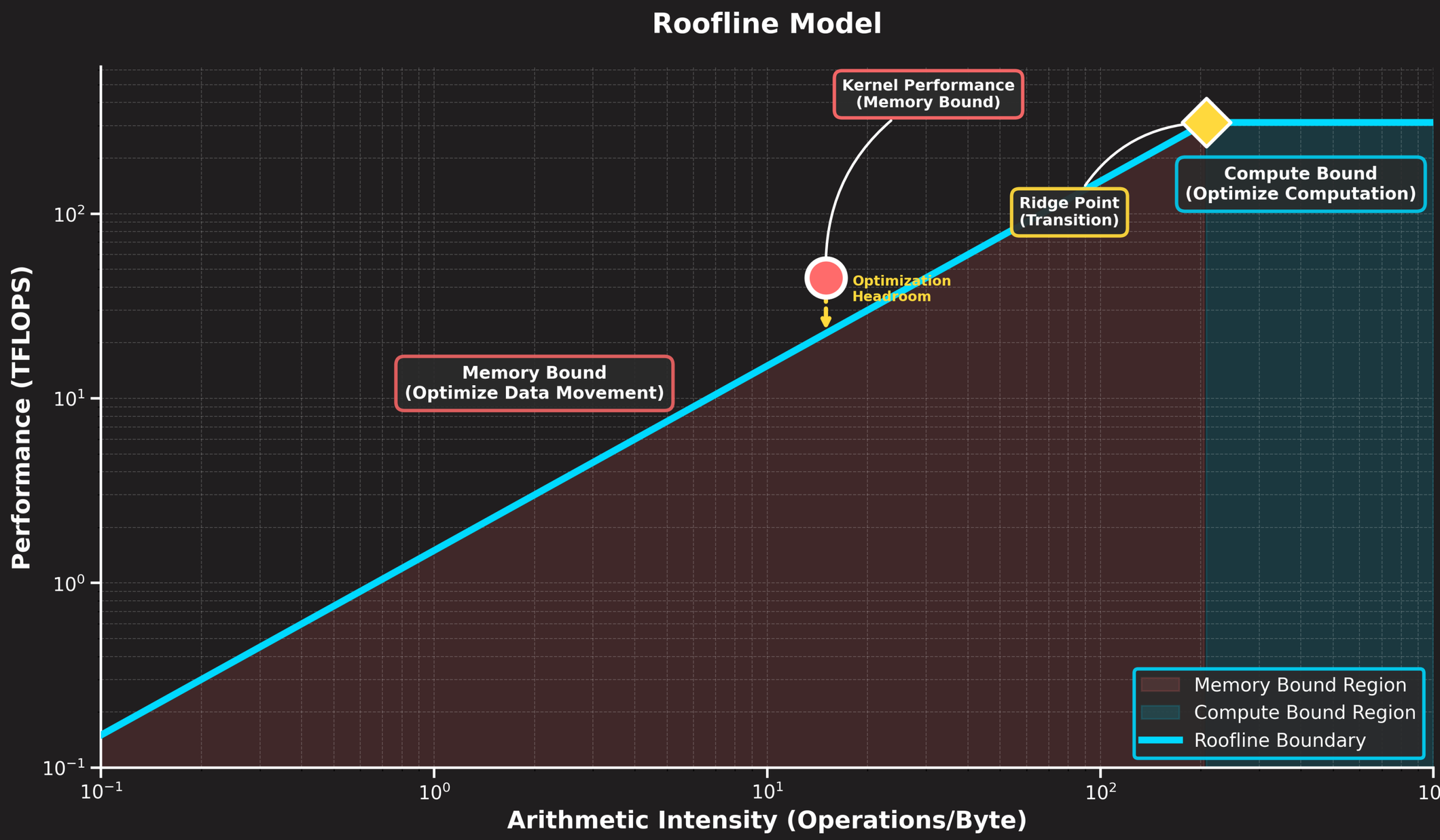
MEMORY-BOUND VS COMPUTE-BOUND

Understanding whether your kernel is compute-bound or memory-bound determines which optimizations will help. There are two scenarios:

✔ If you're **memory-bound** (spending most of your time moving data), increasing compute throughput won't help. **You need to reduce memory traffic (e.g., kernel fusion).**

✔ If you're **compute-bound** (spending most time on FLOPs), optimizing memory access patterns won't help. You need more compute (e.g., upgrade your hardware).

The **roofline model** provides a visual framework for understanding these performance characteristics and identifying optimization opportunities.


Roofline Model
Kernel Performance
(Memory Bound)
Ridge Point
(Transition)
Compute Bound
(Optimize Computation)
Optimization
Headroom
Memory Bound
(Optimize Data Movement)
Performance (TFLOPS)
10^{2}
10^{1}
10^{0}
10^{-1}
10^{-1}
10^{0}
10^{1}
10^{2}
10^{3}
Arithmetic Intensity (Operations/Byte)
Memory Bound Region
Compute Bound Region
Roofline Boundary

POLYGLOT

OUTSIDE A GPU

How GPUs Talk to the World?

COMMUNICATION: CPU ↔ GPU

Now that we know a little of how a GPU performs computation using its internal memory hierarchy, we can address a critical reality: **a GPU doesn't operate in isolation.** Before anything is computed, data must be loaded into the GPU's memory. The CPU needs to schedule kernels and coordinate work. And in distributed training, GPUs must constantly exchange activations, gradients, and model weights.

- ✔ The CPU orchestrates GPU work via PCIe **(Peripheral Component Interconnect Express)** (slow).

However, there are other ways GPUs (within the same host/node) can communicate with each other without going through the slow PCIe.

COMMUNICATION: GPU ↔ GPU

GPUs within a node can communicate in three ways:

- ✔ CPU (slowest, ~3 GB/s, bottlenecked by PCIe)
  >> `NCCL_P2P_DISABLE=1` and `FI_PROVIDER=tcp`

- ✔ GPUDirect RDMA (Remote Direct Memory Access) over EFA NICs (~38 GB/s)
  >> `NCCL_P2P_DISABLE=1` and `FI_PROVIDER=efa`

- ✔ GPUDirect RDMA via NVLink (~786 GB/s bidirectional)
  >> **NCCL automatically prioritizes NVLink when available!**

COMMUNICATION: GPU ↔ GPU

How to know which one I'm using (use `NCCL_DEBUG=INFO`):

- ✔ CPU (slowest, ~3 GB/s, bottlenecked by PCIe)

  >> `NCCL INFO Channel 00 : 1[1] -> 0[0] via SHM/direct/direct`

- ✔ GPUDirect RDMA (Remote Direct Memory Access) over EFA NICs (~38 GB/s)

  >> `…: 1[1] -> 0[0] [receive] via NET/Libfabric/0/GDRDMA/Shared`

- ✔ GPUDirect RDMA via NVLink (~786 GB/s bidirectional)

  >> `NCCL INFO Channel 00/1 : 0[0] -> 1[1] via P2P/CUMEM`

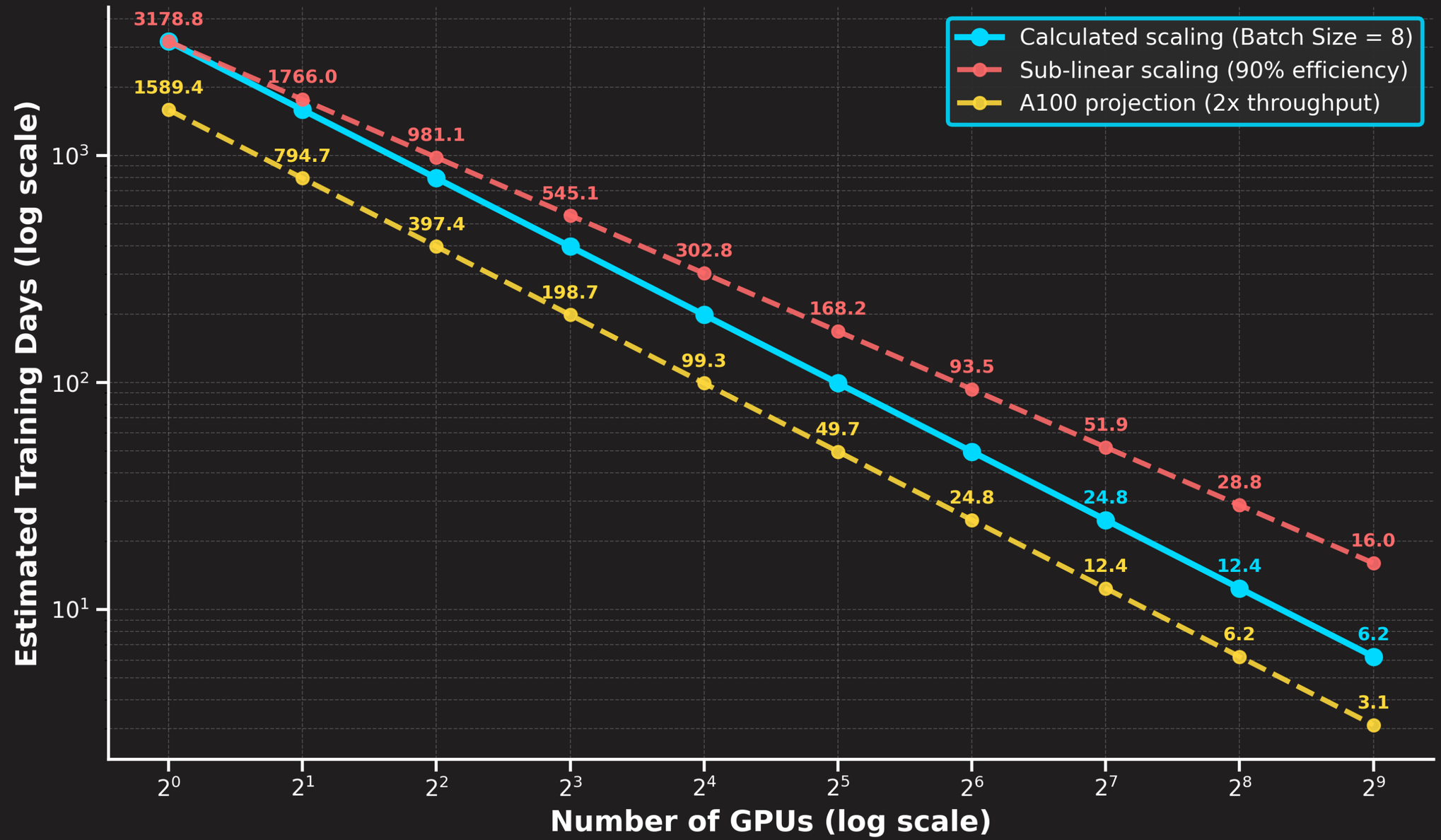
NCCL (NVIDIA Collective Communications Library) Docs

COMMUNICATION: MULTI-NODE

As we scale beyond what a single node can give, training requires **distributing computation across multiple nodes**. There are three main ways in which nodes talk to each other:

- ✔ **Ethernet:** Has evolved from 1 Gbps to 100+ Gbps speeds and remains widely used in HPC
- ✔ **RoCE (RDMA over Converged Ethernet):** Brings RDMA capabilities to Ethernet networks
- ✔ **InfiniBand: 400 Gbps bandwidth, direct GPU-to-GPU memory access** ⚡

Multi-node GPU communication uses **InfiniBand** or **RoCE**. **All-Reduce** scales well (320-350 GB/s stable across nodes). **All-to-all** degrades more sharply due to algorithm complexity. ***Near-constant scaling behavior beyond two nodes when doing distributed data parallel!***


Calculated scaling (Batch Size = 8)
Sub-linear scaling (90% efficiency)
A100 projection (2x throughput)
Estimated Training Days (log scale)
Number of GPUs (log scale)
10^3
10^2
10^1
2^0
2^1
2^2
2^3
2^4
2^5
2^6
2^7
2^8
2^9
3178.8
1766.0
981.1
545.1
302.8
168.2
93.5
51.9
28.8
16.0
1589.4
794.7
397.4
198.7
99.3
49.7
24.8
12.4
6.2
3.1
24.8
12.4
6.2

COMMUNICATION: TROUBLESHOOTING

Almost *nothing* ***works the first try***, and ***debugging*** is part of life. If you're experiencing lower-than-expected bandwidth, systematically check the following areas:

- ✔ **Library versions (CUDA, NCCL)**

  >> Outdated NCCL, EFA, or CUDA libraries may lack critical performance optimizations or bug fixes.

- ✔ **CPU affinity / NUMA**

  >> Improper CPU affinity settings can significantly impact NCCL performance.

- ✔ **Environment variables**

  >> Missing/incorrect environment variables can severely limit bandwidth utilization.

Always try to have good relations with your IT/HPC support.

PLANNING AND HEURISTICS

Tips for Minimizing Troubles on a Training Marathon

POLYGLOT

TRA SUSTAINABLE FUTURES

UNIVERSITÄT BONN

HARDWARE RELIABILITY FOR LONG RUNS

Having enough GPUs is essential for training, but **since LLM training runs can last weeks** and **experiments can last months**, tracking GPU health is critical. Here are some tools to help you assess the integrity of your infrastructure:

- ✔ GPU Fryer → a tool to stress test GPUs and detect throttling or performance degradation.
- ✔ NVIDIA's DCGM diagnostics → a tool to assess cluster readiness levels before a workload is deployed.

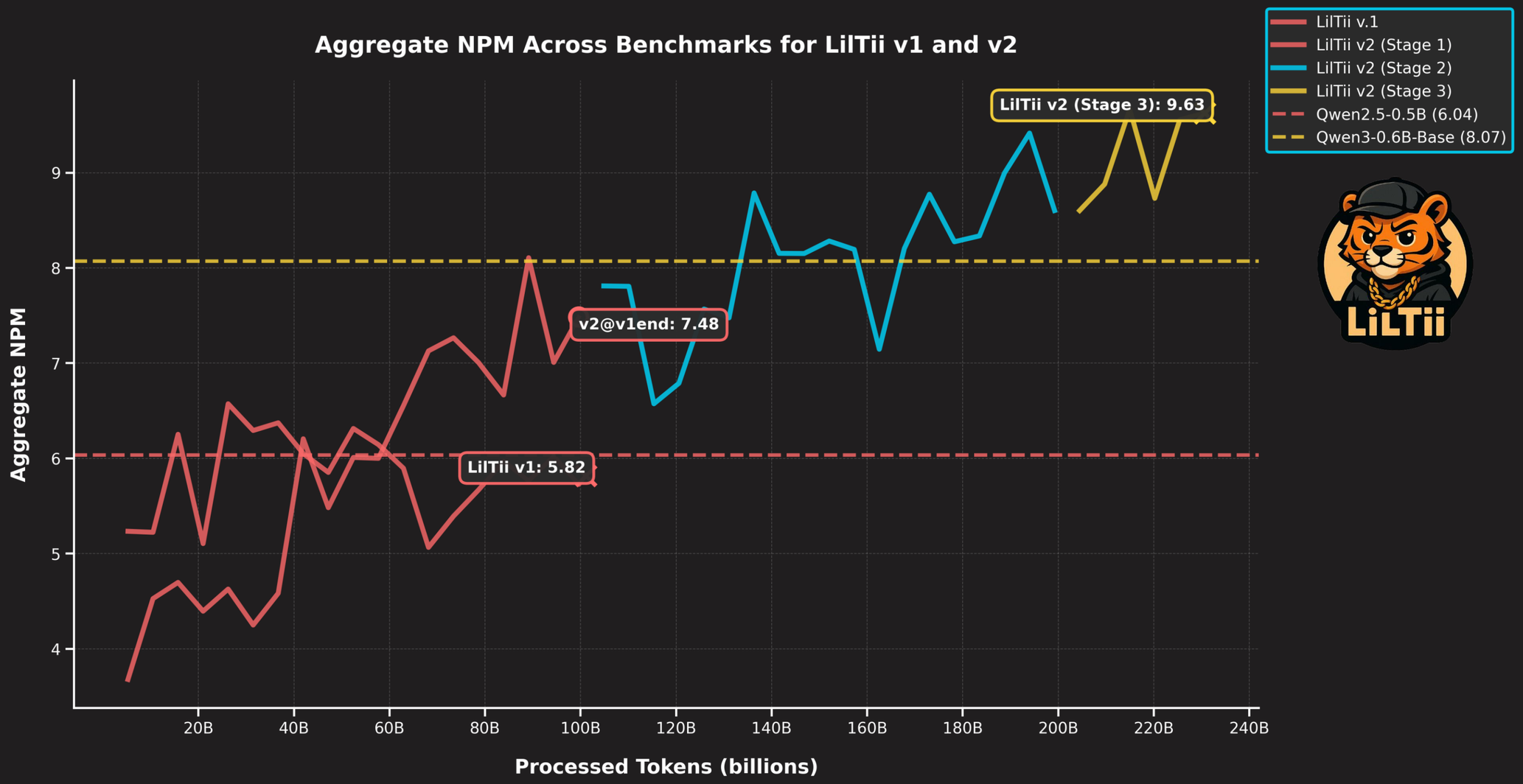
Admins usually do this, but users can also help admins by reporting instances of performance degradation via these tools.

SIZING YOUR TRAINING RUN

After all experiments are done and you are ready to **launch your one-big-run**, you need to determine the **correct number of GPUs** for your case (e.g., *how much time you have? When do you need to publish? etc.*). Here's a simple heuristic you can use:

```
GPU Count = Total FLOPs Required / (Per-GPU Throughput * Target Training Time)
```

- ✔ **Total FLOPs Required** → 6 x 670e+6 x 230e+9 = **9.24e+20**
- ✔ **MFU** → 70%
- ✔ **Per-GPU Throughput** → 300e+12 x 0.70
- ✔ **Target Training Time** → 7 days (604800s)
- ✔ GPU Count: 9.24e+20 / ((300e+12 x 0.70) x 604800) = **~ 8 NVIDIA A100**


Aggregate NPM Across Benchmarks for LilTii v1 and v2
LilTii v.1
LilTii v2 (Stage 1)
LilTii v2 (Stage 2)
LilTii v2 (Stage 3)
Qwen2.5-0.5B (6.04)
Qwen3-0.6B-Base (8.07)
LilTii v2 (Stage 3): 9.63
v2@v1end: 7.48
LilTii v1: 5.82
Aggregate NPM
Processed Tokens (billions)
4
5
6
7
8
9
20B
40B
60B
80B
100B
120B
140B
160B
180B
200B
220B
240B

POLYGLOT

RULES OF THUMB FOR DL ON GPUS

GPUs perform operations efficiently by dividing the work between many parallel processes. Consequently, using parameters that make it easier to break up the operation evenly will lead to the best efficiency.

⚡ The "*Divisible by Powers of Two*" Rule ⚡

- ✔ GPUs looove → 8, 16, 32, **64**, **128**, **256** (larger powers for batch size and hidden dims)
- ✔ Always make/pad your vocab size to a multiple of 8 (TF32 is optimized for multiples of 8)
- ✔ Avoid tiny layers/matrices (<128)
- ✔ Watch for wave quantization effects
- ✔ Tensor shapes = micro-optimizations
- ✔ If you must, *go for something that has only 3 as an odd factor!*

Must Read: NVIDIA Deep Learning Performance

FINAL THOUGHTS

1. Doing research with LLMs without HPC severely limits what you can do.

2. Having a systems-level understanding of HPC — more on that with our following tutor — will significantly help you solve the unavoidable problems you will have.

3. If you are serious about specializing in this field, you should study HPC-related topics (e.g., how computers work, learn a systems-level programming language).

THANK YOU

TRA SUSTAINABLE FUTURES

UNIVERSITÄT BONN